Architecture notes review

arch_1_directions.md

- The headline table mixes runs across three taxonomies (merged_25, une_merge_v1, une_merge_v1_nosides). The "comparability caveats" flag this in prose just above the table, but the rows aren't visually separated. A hard rule between groups, or a column showing active class count, would stop accidental cross-taxonomy comparisons mid-skim.
- The wipe_drop win (run_20260503_172922) reports "+0.5 macro / +1.2 min ws" over the loss-side best. Macro lift is well above the 0.1% envelope. The +1.2 ws is inside the ~2 pp run-mean envelope. The corresponding +1.5 smash lift sits in a per-class envelope wider than 2 pp (augmentation_framework's "Seed-noise envelope" section reports per-class per-serial spreads up to 11 pp on cross_court_net_shot). So neither half of the smash/ws lift is statistically carried by its magnitude alone. The both-pair-up qualitative pattern (vs CDB pair-trade) and the per-clip mid-clip zeroing mechanism in frame_zeroing.md carry it.
- arch_1's TLDR is dated 2026-05-04 but the doc body has a "2026-05-06 Aug v1 round 1 hparam sweep" entry at line 437 with full run results. Active priority #2 ("Augmentation set landing") still reads as pending in the TLDR while the body shows the locked aug v1 + p_jitter=0.3 already ran four sweep cells on 2026-05-05/06. The spine doc is stale against its own body, not just against augmentation_framework.
- Two baselines, second undefined in arch_1: line 12 calls run_20260503_172922 "active"; line 439 names run_20260505_154907 as the aug-side reference (no headline-table row, no timeline entry). hparam_search_wrapper.md formalises this as two distinct roles: `current_best_run` (movable, run_20260505_154907) vs `wipe_drop_best_run` (frozen reference, run_20260503_172922). arch_1's "active baseline" label conflates the roles, and per the wrapper run_20260505_154907 is the live current-best, not the run arch_1's TLDR holds up. Adopt the wrapper's two-role naming.
- TLDR alone is ~50 lines (headline + caveats + key learnings + active priorities). The "spine, referenced from everywhere" role and the "working notebook for current state" role are pulling in different directions. Possible to extract a thin current-state doc and let arch_1 carry the longer narrative; not urgent.
- TrackNetV3-inpaint caveat: our reproduction used the inpaint variant from day one, BST paper didn't. Some of the "we beat BST paper" lift is the cleaner shuttle data.

frame_zeroing.md

- "Outcome update" box at the top of the doc (Step 1 lifted, Step 2 didn't) is excellent practice. Other working docs would benefit from the same pattern: outcome stamped at the top, body explains the design.
- The 0c analysis was reversed on smash/ws specifically by the 2026-05-03 per-clip re-cut. Original ±10-of-hit framing showed bottleneck classes as "shuttle reliably present"; the per-clip re-cut found mid-clip zeroing of 13.7% (smash) and 8.5% (ws). The doc handles this honestly. Methodology lesson worth carrying into future audits: windowed analyses can hide signal at a level dependent on window choice.

- Mechanism story for wipe_drop is well-traced. Two candidate mechanisms named (coarse-class prior vs shuttle-data-quality asymmetry); the both-pair-up result supports mechanism 2. CDB-F1's pair-trade pattern gave the contrast. Watch out though: frame_zeroing:385 quotes the trade as smash -5.5 / ws +8.7 as if vs the same baseline, but per arch_1:303 the +8.7 is vs LS=0.1 baseline while the -5.5 is per-class shift vs class-weighted run. Apples-to-apples vs class-weighted is ws +4.0 / smash -5.5. Qualitative pair-trade pattern still holds; the magnitude is overstated in the source.
- Variant 2a "tested and dropped" is well-documented and the rationale is right (model already inferring missing-shuttle from xy + temporal context, plus the new fuse layer ate budget for a near-identity solution). 2b's deferral is correctly reasoned (thinner per-frame signal than 2a, unlikely to lift if 2a didn't).
- One residual question: the mechanism analysis says mechanism 2 is "real" because both pair members lifted, but doesn't quantify how much each mechanism contributes. CDB-F1's pair-trade attacks mechanism 1; wipe_drop's both-up suggests mechanism 2 is also live. Whether mechanism 1 is residual after wipe_drop, or whether wipe_drop saturates the pair gap, isn't pinned down. Bears on whether further per-class alpha tuning has room.

augmentation_framework.md

- "Broken but regularising" finding: turning off the conceptually-wrong RandomTranslation_batch regressed macro -0.8 / min -4.4 / acc -0.7 vs wipe_drop best. Plan is "replace, don't disable". The min -4.4 is well above the ~2 pp envelope so the result is real signal. Worth being explicit that the current best run shipped with the broken aug enabled, so the locked Task 2 set isn't strictly "current best plus a fix"; it could move either way.
- Seed envelope claim ("~0.1% on macro/acc/top-2 and ~2% on min F1") is the load-bearing number for triaging signal vs noise across the whole project. The "Seed-noise envelope on run means" section gives the source (identical-config replicate from the aug v1 round 1 sweep) and reports per-class per-serial spreads up to 11 pp. Worth noting in the TLDR that the 2 pp envelope is run-mean, not per-class per-serial; per-class lift claims need a wider envelope.
- "Locked Task 2 set (2026-05-04)" reads ambiguous: locked-as-decision, or locked-as-shipped? arch_1's active priorities #2 is "Augmentation set landing", so it sounds decision-locked but not yet in a training run. Worth being explicit.
- Cross-doc drift on Method A vs B: this doc's TLDR frames Method A as faithful-to-annotation primary, Method B as independent verification. x3d Stage 1's "working hypothesis (revised)" suggests Method B may end up primary instead. Status needs reconciling.
- Coordinate-spaces verification (three streams, three frames) with file:line traces is a good pattern. Worth keeping consistent across the rest of the doc.
- Doc is 1272 lines. Not flagging as a problem now, but if Phase 3 candidates and X3D-S derivations grow further inside it, another extraction may be warranted.

x3d_integration_macro_plan.md

- 6 stages plus Stage 0 pre-flight, each with Goal / Open questions / Deliverable / Dependencies. Good template, easy to read.

- Each stage carries 5-15+ open questions with no within-stage gating. For a macro plan this is fine, but when each stage becomes an executable task the gates need flagging, or the per-stage task balloons. Worth marking "decision-gate" vs "nice-to-have" before stages get spun out.
- Stage 1 working hypothesis revision (Method B may be primary) hasn't propagated back to augmentation_framework's TLDR. Cross-doc drift to resolve.
- Stage 0 pre-flight: "fusion baseline is whatever's best after Capacity Run 2 + aug land". So the X3D-S fusion A/B is against a baseline that doesn't exist yet.
- Solo X3D-S baseline is gated as "lower bound for fusion adding something". Edge case the plan doesn't call out: what if solo X3D-S beats BST on the smash/wrist_smash pair? The plan reads as if fusion is presumed. The off-ramp ("solo X3D-S is enough") isn't named.
- Plan deliberately defers Capacity Run 2 and augmentation landing (out of scope, queued in arch_1). Worth a one-line dependency-graph in Context that maps each prerequisite to its canonical doc, so the reader doesn't have to hunt.

hparams_sweep_speculations.md

- TLDR's "likeliest single-knob lever after the loss / class-weighting work finishes: weight decay" is probably stale. arch_1 now describes loss-side as thoroughly explored, and the live queue is Cap Run 2 + aug + X3D-S. Weight decay isn't visible in arch_1's active priorities. Worth refreshing or re-scoping.
- Geometry section says (DL=100, DA=128) is validated and the only sane move is "up the curve". This is now arch_1's active priority #1 (d_model 100→192). TLDR pre-dates this being live; small refresh.
- "BST paper inferences worth acting on": variable-length clipping carries a +5.7 pp prior on min-F1 from the paper's own ablation. At the current ws floor of ~0.47, a +5.7 pp prior dwarfs every loss-side knob and the wipe_drop fix combined. The doc calls it "the cheapest possible experiment" yet it's not visibly in the live queue or the parked-decisions list. Either run it or document why not. The magnitude warrants a stronger flag than "has it been run".
- "Frame-zeroing redesign extracted to frame_zeroing.md, stub remains here as research-direction pointer" - the stub-vs-canonical relationship isn't obvious from the TLDR. Minor friction for a second-eye reader.
- Run-budget estimate (~105-140 A100-hours, two weeks of overnight runs) pre-dates the wipe_drop finding and X3D-S being prioritised. Composition likely stale; rough scale still useful.
- With the queue shifting toward signal-side (X3D-S) and encoder capacity, this doc's role may need to shrink to weight-decay + dropout cells, with the rest parked or migrated to arch_1. Worth a scope-freeze review.

train_val_test_split_analysis.md

- Train F1 is the in-training running-argmax metric, not a held-out eval. Methodology note flags this honestly, but the implication isn't drawn out: train F1 is on augmented data with rolling model state, val F1 is held-out clean data with fixed state. The qualitative conclusion (large train-test gap on confusion pair) is robust, but the magnitude is probably overstated by a couple of pp. Cheap fix: a single eval-mode pass over training data per checkpoint would settle it. Worth flagging because "6-9 pp train headroom" is doing real load-bearing work in the X3D-S argument.

- The doc emphasises label-noise as the gap mechanism. The player-overlap diagnostic at class_player_split_overlap_exploration.md gives the canonical numbers: train-val 55% / train-test 15% / val-test 21% clip-weighted (and 45% / 8% / 5% Jaccard). val and test have very different player-overlap profiles with train, so the val-test gap has a real player-shift component independent of label noise. Worth cross-linking into this doc to inform the swap-val-test reframe.
- Val-test gap chart text says "consistently ~3-4 pp across all runs"; the per-run table actually shows 2.5-3.3 pp. Loose framing - call it ~3 pp or ~2.5-3.5 pp to match the data.
- Saved-checkpoint vs end-of-training epochs are mixed across the doc. Headline says "6-9 pp train headroom" (end of training). Per-run table and per-class table are both at saved-checkpoint (where the gap is 1-3 pp on macro). Both numbers are correct, but the reader has to track which epoch each claim is at. The cRT/Kang argument is cleanest at saved-checkpoint anyway, so that's a reasonable anchor for the headline.
- Per-class breakdown is one run (CDB τ1γ1, "the winner"). Reasonable choice, but a 5-run mean across all CDB runs would harden the three-group observation against single-run noise.
- cRT/LWS proposal is the strongest concrete next step in this doc (freeze encoder, retrain head with class-balanced sampling). Doc says "Project decision pending" without naming what's pending. Worth being explicit: competing with X3D-S Stage 1 work for time, waiting on a free GPU, or waiting on someone to volunteer? If it's cheap enough to slot in before X3D-S, the encoder-vs-classifier-bottleneck answer is useful for X3D-S design.

model_capacity_bottleneck_question.md

- Argument is thoroughly scaffolded: EMC / double descent (Nakkiran, Belkin), Kaplan and Chinchilla scaling laws, skeleton-AR field reference points (1-3M cluster on NTU), Voita head over-provisioning, video-AR small-data literature (X3D-M / MoViNet 3-5M cluster on Kinetics-400 from-scratch). Concludes the plateau is data-bound and signal-bound, not capacity-bound. Gold-standard documentation for a research decision; nothing to add to the argument itself.
- Player-overlap finding cites val-test 5% at line 155. The source actually reports val-test 5% Jaccard but 21% clip-weighted; the 5% in this doc is the Jaccard, used in the same sentence as the 55% train-val number which is clip-weighted. Mixed metrics in one statement. Either name both metrics, or pick one consistently — clip-weighted is the more useful framing for "how much of the data has player overlap".
- "Smallest-risk capacity options" table explicitly frames Cap Run 2's expected gain as 0-2 pp. transformer_widening_hparam_changes.md echoes this ("expected gain on test macro 0-2 pp, possibly less"). So Cap Run 2 is the deliberate close-out-the-question move with a weak prior, not a contradiction with the data-bound conclusion.
- One internal observation worth verbalising: line 289 says "more capacity makes [player-overlap] memorisation room larger, not smaller". By that argument, Cap Run 2 could regress val (more memorisation of player-specific cues at the 55% overlap) even if test stays flat. The prediction is testable cheaply by reading val vs test movement separately on Run 2: val should regress if the prediction holds, test is the headline. Worth pre-committing to that diagnostic before launch so the result is interpretable either way.

- The doc's conclusion is consistent with train_val_test_split_analysis's conclusion. Both point to data + signal as the bottleneck, both name X3D-S fusion + cRT as the next moves. Cross-doc consistency is good; an explicit handshake at the top of each doc ("see X for the parallel plateau analysis") would make the spine tighter.

Cross-doc / process

- 27 .md files at the folder root with no INDEX.md or active/superseded/background tagging is hard to second-eye. Suggest adding an INDEX.md (active spine docs, current-work docs, parked, archived) plus a one-line status header inside each doc (active / superseded / archived; last touched). Cheap, large readability win.
- Player-overlap finding is cited in arch_1 (lines 469, 514) and capacity_bottleneck (line 155). Source (class_player_split_overlap_exploration.md, lines 154-160 + 124-126): train-val 55% / train-test 15% / val-test 21% clip-weighted; 45% / 8% / 5% Jaccard. arch_1:469 is correct. arch_1:514 has a label typo — the "val-test 15%" is actually train-test 15%. capacity_bottleneck:155 silently mixes metrics — its "55% / 5%" pair is clip-weighted then Jaccard in one sentence. Pick one metric (clip-weighted preferred), state all three pair numbers consistently, propagate to both citing docs.
- Method A vs B status drift between augmentation_framework TLDR and x3d Stage 1 is the next clearest cross-doc inconsistency. Resolve in one place, point the other at it.
- nosides_runs_table.md stops at run_20260502_075808 (CDB γ=2). Wipe_drop, mask 2a, Cap Run 1, jitter-off, and four aug v1 round 1 sweep cells aren't in there. Doc's loss-config column scopes it as loss-side, which excuses the data and capacity runs but not the four aug-side cells. Either update the table or scope-narrow it explicitly.
- Date-stamped TLDRs are great where they exist (arch_1 says "2026-05-04", augmentation_framework section says "2026-05-06", frame_zeroing has a 2026-05-03 outcome stamp). Not consistent across docs, and arch_1's date is now stale against its own body. A standard "last updated YYYY-MM-DD" in every doc's first heading would let a reader spot stale spine doc against fresh working doc instantly.